

- **University Name: Charutar Vidya Mandal University (CVMU)**
- **Collage Name: A.D. Patel Institute of Technology (ADIT)**
- **Course Title & code: Programing with JAVA (202044502)**
- **Assignment Number: Assignment 1**
- **Student Name: Patel Yashkumar Shaileshkumar**
- **Enrollment Number: 12402080601158**
- **Semester & academic Year: sem-4 (2025-26)**
- **Github Link: <https://github.com/yash-1158/Java-Programming-Assignment>**

Assignment 1

➤ Program 1

Problem Statement:

Write a Java program to perform basic Array and String operations such as Reverse, Sort, Search, Average, and Maximum using class and objects.

Objective:

To develop a Java program that performs basic Array and String operations such as Reverse, Sort, Search, Average, and Maximum using class and objects.

Source Code:

```
import java.util.Scanner;
class Operations {
    void revString(String str) {
        String rev = "";
        for (int i = str.length() - 1; i >= 0; i--) {
            rev = rev + str.charAt(i);
        }
        System.out.println("Reversed String: " + rev);
    }
    void revArray(int arr[]) {
        int n = arr.length;
        for (int i = 0; i < n / 2; i++) {
            int temp = arr[i];
            arr[i] = arr[n - 1 - i];
            arr[n - 1 - i] = temp;
        }
        System.out.print("Reversed Array: ");
        for (int i = 0; i < n; i++) {
            System.out.print(arr[i] + " ");
        }
        System.out.println();
    }
    void sortArray(int arr[]) {
        int n = arr.length;
        for (int i = 0; i < n - 1; i++) {
            for (int j = 0; j < n - 1 - i; j++) {
                if (arr[j] > arr[j + 1]) {
                    int temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                }
            }
        }
        System.out.print("Sorted Array: ");
        for (int i = 0; i < n; i++) {
            System.out.print(arr[i] + " ");
        }
        System.out.println();
    }
    void searchArray(int arr[], int key) {
        boolean found = false;
        for (int i = 0; i < arr.length; i++) {
            if (arr[i] == key) {
                System.out.println("Element found at index: " + i);
            }
        }
    }
}
```

Name : Patel Yashkumar Shaileshkumar

Enrollment No : 12402080601158

```
for (int i = 0; i < arr.length; i++) {
    if (arr[i] == key) {
        found = true;
        break;
    }
}
if (!found) {
    System.out.println("Element not found");
}
}

void avgArray(int arr[]) {
    int sum = 0;
    for (int i = 0; i < arr.length; i++) {
        sum += arr[i];
    }
    double avg = (double) sum / arr.length;
    System.out.println("Average: " + avg);
}

void maxArray(int arr[]) {
    int max = arr[0];
    for (int i = 1; i < arr.length; i++) {
        if (arr[i] > max) {
            max = arr[i];
        }
    }
    System.out.println("Maximum Element: " + max);
}

}

public class Program1 {
    public static void main(String[] args) {

        Scanner input = new Scanner(System.in);
        Operations a = new Operations();

        System.out.println("-> String Opreations\n");
        System.out.print("Enter String: ");
        String str = input.nextLine();
        a.revString(str);

        System.out.println("\n-> Array Opreations\n");
        System.out.print("Enter number of elements: ");
```

```
int n = input.nextInt();
int arr[] = new int[n];

System.out.println("Enter array elements:");
for (int i = 0; i < n; i++) {
    arr[i] = input.nextInt();
}

a.revArray(arr);
a.sortArray(arr);

System.out.print("Enter element to search: ");
int key = input.nextInt();
a.searchArray(arr, key);

a.avgArray(arr);
a.maxArray(arr);

input.close();
}
```

Output:

```
-> String Opreations

Enter String: yash
Reversed String: hsay

-> Array Opreations

Enter number of elements: 2
Enter array elements:
1
12
Reversed Array: 12 1
Sorted Array: 1 12
Enter element to search: 12
Element found at index: 1
Average: 6.5
Maximum Element: 12
```

➤ Program 2

Problem Statement:

Develop Matrix class with constructors, transpose and multiplication.

Objective:

To develop a Java program using class and objects that performs basic operations on arrays and strings, such as reversing elements, sorting the array, searching for a specific element, calculating the average, finding the maximum value, and reversing a given string.

Source Code:

```
2 import java.util.Scanner;
3 class Matrix {
4     int row, col;
5     int arr[][];
6     Matrix(int r, int c) {
7         row = r;
8         col = c;
9         arr = new int[row][col];
10    }
11    void readMatrix(Scanner input) {
12        for (int i = 0; i < row; i++) {
13            for (int j = 0; j < col; j++) {
14                arr[i][j] = input.nextInt();
15            }
16        }
17    }
18    void displayMatrix() {
19        for (int i = 0; i < row; i++) {
20            for (int j = 0; j < col; j++) {
21                System.out.print(arr[i][j] + " ");
22            }
23            System.out.println();
24        }
25    }
26    Matrix transpose() {
27        Matrix t = new Matrix(col, row);
28
29        for (int i = 0; i < row; i++) {
30            for (int j = 0; j < col; j++) {
31                t.arr[j][i] = arr[i][j];
32            }
33        }
34        return t;
35    }
36    Matrix multiply(Matrix m) {
37        if (this.col != m.row) {
38            System.out.println("Matrix multiplication not possible");
39            return null;
40        }
41        Matrix result = new Matrix(this.row, m.col);
42        for (int i = 0; i < this.row; i++) {
43            for (int j = 0; j < m.col; j++) {
44                for (int k = 0; k < this.col; k++) {
45                    result.arr[i][j] += this.arr[i][k] * m.arr[k][j];
46                }
47            }
48        }
49        return result;
50    }
51 }
```

Name : Patel Yashkumar Shaileshkumar

Enrollment No : 12402080601158

```
public class Program2 {  
    public static void main(String[] args) {  
  
        Scanner input = new Scanner(System.in);  
  
        System.out.print("Enter rows and columns of Matrix 1: ");  
        int r1 = input.nextInt();  
        int c1 = input.nextInt();  
  
        Matrix m1 = new Matrix(r1, c1);  
        System.out.println("Enter elements of Matrix 1:");  
        m1.readMatrix(input);  
  
        System.out.print("Enter rows and columns of Matrix 2: ");  
        int r2 = input.nextInt();  
        int c2 = input.nextInt();  
  
        Matrix m2 = new Matrix(r2, c2);  
        System.out.println("Enter elements of Matrix 2:");  
        m2.readMatrix(input);  
  
        System.out.println("\nMatrix 1:");  
        m1.displayMatrix();  
  
        System.out.println("\nMatrix 2:");  
        m2.displayMatrix();  
  
        System.out.println("\nTranspose of Matrix 1:");  
        Matrix t1 = m1.transpose();  
        t1.displayMatrix();  
  
        System.out.println("\nTranspose of Matrix 2:");  
        Matrix t2 = m2.transpose();  
        t2.displayMatrix();  
  
        System.out.println("\nMatrix Multiplication Result: ");  
        Matrix mul = m1.multiply(m2);  
        if (mul != null) {  
            mul.displayMatrix();  
        }  
  
        input.close();  
    }  
}
```

Output :

```
Enter rows and columns of Matrix 1: 2  
1  
Enter elements of Matrix 1:  
1  
2  
Enter rows and columns of Matrix 2: 1  
2  
Enter elements of Matrix 2:  
2  
2  
  
Matrix 1:  
1  
2  
  
Matrix 2:  
2 2  
  
Transpose of Matrix 1:  
1 2  
  
Transpose of Matrix 2:  
2  
2  
  
Matrix Multiplication Result:  
2 2  
4 4
```


➤ Program 3

Problem Statement :

Write a program demonstrating Wrapper classes and String vs StringBuffer

Objective:

To develop a Java program that demonstrates the use of wrapper classes and illustrates the difference between String and StringBuffer, highlighting their behavior and basic operations in Java.

Source Code :

```
3  import java.util.Scanner;
4
5  public class Program3 {
6
7      public static void main(String[] args) {
8
9          Scanner sc = new Scanner(System.in);
10
11          System.out.println("Wrapper Class\n");
12          System.out.print("Enter an integer: ");
13          int num = sc.nextInt();
14
15          Integer objNum = num;
16          System.out.println("Integer object: " + objNum);
17
18          int unboxedNum = objNum;
19          System.out.println("primitive int: " + unboxedNum);
20
21          System.out.println("\nString vs StringBuffer");
22          sc.nextLine();
23
24          System.out.print("\nEnter a String: ");
25          String str = sc.nextLine();
26
27          System.out.print("Enter word to concat: ");
28          String concatWord = sc.nextLine();
29
30          str.concat(concatWord);
31          System.out.println("String after concat: " + str);
32
33          System.out.print("Enter a StringBuffer value: ");
34          StringBuffer sb = new StringBuffer(sc.nextLine());
35
36          System.out.print("Enter word to append: ");
37          String appendWord = sc.nextLine();
38
39          sb.append(appendWord);
40          System.out.println("StringBuffer after append: " + sb);
41
42          sc.close();
43      }
44 }
```

Name : Patel Yashkumar Shaileshkumar

Enrollment No : 12402080601158

OutPut:

```
Wrapper Class

Enter an integer: 12
Integer object: 12
primitive int: 12

String vs StringBuffer

Enter a String: Yash
Enter word to concat: Dhyey
String after concat: Yash
Enter a StringBuffer value: Yash
Enter word to append: Patel
StringBuffer after append: YashPatel
```


➤ Program 4

Problem Statement :

Implement BankAccount class with deposit, withdraw and balance inquiry

Objective:

To develop a Java program that implements a BankAccount class and performs basic banking operations such as depositing money, withdrawing money, and checking the account balance using appropriate methods.

Source Code :

```
import java.util.Scanner;
class BankAccount {
    private String name;
    private int accNo;
    private double balance;
    private static int nextAccNo = 1;
    public BankAccount(String name) {
        this.name = name;
        this.accNo = nextAccNo++;
        this.balance = 0;
    }
    public int getAccNo() {
        return accNo;
    }
    public void deposit(double amt) {
        if (amt <= 0) {
            System.out.println("Invalid amount! Deposit must be positive.");
        } else {
            balance += amt;
            System.out.println(amt + " deposited successfully.");
        }
    }
    public void withdraw(double amt) {
        if (amt <= 0) {
            System.out.println("Invalid amount! Withdraw must be positive.");
        } else if (amt > balance) {
            System.out.println("Insufficient balance!");
        } else {
            balance -= amt;
            System.out.println(amt + " withdrawn successfully.");
        }
    }
    public void showDetails() {
        System.out.println("Account No: " + accNo + " | Name: " + name + " | Balance: " + balance);
    }
}

public class Program4 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        BankAccount[] accounts = new BankAccount[10];
        int count = 0;
    }
}
```

Name : Patel Yashkumar Shaileshkumar

Enrollment No : 12402080601158

```
while (true) {
    System.out.println("\n=====");
    System.out.println("1.Create  2.Deposit  3.Withdraw");
    System.out.println("4.Show    5.Exit");
    System.out.println("=====");
    System.out.print("Choose option: ");
    if (!sc.hasNextInt()) {
        System.out.println("Invalid choice! Enter number only.");
        sc.next();
        continue;
    }
    int ch = sc.nextInt();
    switch (ch) {
        case 1:
            if (count >= 10) {
                System.out.println("Bank full!");
                break;
            }
            sc.nextLine();
            System.out.print("Enter name: ");
            String name = sc.nextLine();
            accounts[count] = new BankAccount(name);
            System.out.println("Account created successfully!");
            accounts[count].showDetails();
            count++;
            break;
        case 2:
            System.out.print("Enter account number: ");
            if (!sc.hasNextInt()) {
                System.out.println("Invalid account number!");
                sc.next();
                break;
            }
            int accNo = sc.nextInt();
            BankAccount acc = find(accounts, count, accNo);
            if (acc == null) {
                System.out.println("Account not found!");
                break;
            }
            System.out.print("Deposit amount: ");
            if (!sc.hasNextDouble()) {
```

```
case 3:
    accNo = sc.nextInt();
    acc = find(accounts, count, accNo);
    if (acc == null) {
        System.out.println("Account not found!");
        break;
    }
    System.out.print("Withdraw amount: ");
    if (!sc.hasNextDouble()) {
        System.out.println("Invalid amount!");
        sc.next();
        break;
    }
    double w = sc.nextDouble();
    acc.withdraw(w);
    break;
case 4:
    System.out.print("Enter account number: ");
    if (!sc.hasNextInt()) {
        System.out.println("Invalid account number!");
        sc.next();
        break;
    }
    accNo = sc.nextInt();
    acc = find(accounts, count, accNo);
    if (acc == null) {
        System.out.println("Account not found!");
    } else {
        acc.showDetails();
    }
    break;
case 5:
    System.out.println("Thank you for using Bank System.");
    sc.close();
    System.exit(0);
default:
    System.out.println("Invalid choice!");
}
}

static BankAccount find(BankAccount[] accounts, int count, int accNo) {
    for (int i = 0; i < count; i++) {
        if (accounts[i].getAccNo() == accNo) {
            return accounts[i];
        }
    }
    return null;
}
}
```

OutPut :

```
=====
1.Create  2.Deposit  3.Withdraw
4.Show    5.Exit
=====
Choose option: 1
Enter name: yash
Account created successfully!
Account No: 1 | Name: yash | Balance: 0.0

=====
1.Create  2.Deposit  3.Withdraw
4.Show    5.Exit
=====
Choose option: 2
Enter account number: 1
Deposit amount: 100
100.0 deposited successfully.

=====
1.Create  2.Deposit  3.Withdraw
4.Show    5.Exit
=====
Choose option: 3
Enter account number: 1
Withdraw amount: 50
50.0 withdrawn successfully.

=====
1.Create  2.Deposit  3.Withdraw
4.Show    5.Exit
=====
Choose option: 4
Enter account number: 1
Account No: 1 | Name: yash | Balance: 50.0

=====
1.Create  2.Deposit  3.Withdraw
4.Show    5.Exit
=====
Choose option: 5
Thank you for using Bank System.
```

➤ Program 5

Problem Statement :

Develop inheritance-based Cricket–Match system using command line arguments

Objective:

To develop a Java program that demonstrates the concept of inheritance by creating a Cricket Match system and using command line arguments to pass match details such as team names, runs, or overs, and display the match information through inherited classes.

Source Code :

```
class Match {
    String team1, team2;

    Match(String t1, String t2) {
        team1 = t1;
        team2 = t2;
    }

    void displayTeams() {
        System.out.println("Team 1 : " + team1);
        System.out.println("Team 2 : " + team2);
    }
}

class CricketMatch extends Match {
    int score1, score2;

    CricketMatch(String t1, String t2, int s1, int s2) {
        super(t1, t2);
        score1 = s1;
        score2 = s2;
    }

    void displayScore() {
        System.out.println(team1 + " Score : " + score1);
        System.out.println(team2 + " Score : " + score2);
    }

    void showResult() {
        if (score1 > score2)
            System.out.println("Winner : " + team1);
        else if (score2 > score1)
            System.out.println("Winner : " + team2);
        else
            System.out.println("Match Result : Draw");
    }
}
```



```
public class Program5 {  
    public static void main(String args[]) {  
  
        if (args.length < 4) {  
            System.out.println("Please enter arguments:");  
            System.out.println("Example: java Program5 India Australia 250 230");  
            return;  
        }  
  
        String team1 = args[0];  
        String team2 = args[1];  
        int score1 = Integer.parseInt(args[2]);  
        int score2 = Integer.parseInt(args[3]);  
  
        CricketMatch match = new CricketMatch(team1, team2, score1, score2);  
  
        System.out.println("==== CRICKET MATCH RESULT =====");  
        match.displayTeams();  
        System.out.println("-----");  
        match.displayScore();  
        System.out.println("-----");  
        match.showResult();  
        System.out.println("=====");  
    }  
}
```

Output:

```
C:\Users\dell\Desktop\Java-Programing-Assignment\Assignment-1\Program-5>java Program5  
Please enter arguments:  
Example: java Program5 India Australia 250 230  
  
C:\Users\dell\Desktop\Java-Programing-Assignment\Assignment-1\Program-5>java Program5 India Australia 250 230  
==== CRICKET MATCH RESULT =====  
Team 1 : India  
Team 2 : Australia  
-----  
India Score : 250  
Australia Score : 230  
-----  
Winner : India  
=====
```


➤ Program 6

Problem Statement :

Implement Cipher system using Abstract class and method overriding

Objective:

To develop a Java program that implements a Cipher system using an abstract class and method overriding to perform encryption and decryption operations.

Source Code :

```
import java.util.Scanner;
abstract class Cipher {
    abstract String encrypt(String text);
    abstract String decrypt(String text);
}
class SimpleCipher extends Cipher {
    public String encrypt(String text) {
        String result = "";
        for (int i = 0; i < text.length(); i++) {
            char ch = text.charAt(i);
            ch = (char)(ch + 3);
            result += ch;
        }
        return result;
    }
    public String decrypt(String text) {
        String result = "";
        for (int i = 0; i < text.length(); i++) {
            char ch = text.charAt(i);
            ch = (char)(ch - 3);
            result += ch;
        }
        return result;
    }
}
public class Program6 {
    public static void main(String args[]) {
        Scanner sc = new Scanner(System.in);
        System.out.println("===== SIMPLE CIPHER SYSTEM =====\n");

        System.out.print("Enter Original Text : ");
        String text = sc.nextLine();
        SimpleCipher obj = new SimpleCipher();
        String encrypted = obj.encrypt(text);
        String decrypted = obj.decrypt(encrypted);

        System.out.println("-----");
        System.out.println("Encrypted Text : " + encrypted);
        System.out.println("Decrypted Text : " + decrypted);
        System.out.println("-----");
    }
}
```

OutPut :

```
===== SIMPLE CIPHER SYSTEM =====

Enter Original Text : hello
-----
Encrypted Text : khood
Decrypted Text : hello
=====
```

Name : Patel Yashkumar Shaileshkumar

Enrollment No : 12402080601158

➤ Program 7

Problem Statement :

Demonstrate Inner Classes (member, local, anonymous)

Objective:

To develop a Java program that demonstrates the use of different types of inner classes such as member inner class, local inner class, and anonymous inner class and understand their implementation and usage in Java.

Source Code :

```
class Outer {
    class MemberInner {
        void show() {
            System.out.println("This is Member Inner Class");
        }
    }
    void display() {
        class LocalInner {
            void show() {
                System.out.println("This is Local Inner Class");
            }
        }
        LocalInner l = new LocalInner();
        l.show();
    }
}

interface Test {
    void show();
}

public class Program7 {
    public static void main(String args[]) {
        System.out.println("===== INNER CLASS DEMONSTRATION =====");
        Outer o = new Outer();
        Outer.MemberInner m = o.new MemberInner();
        m.show();
        o.display();
        Test t = new Test() {
            public void show() {
                System.out.println("This is Anonymous Inner Class");
            }
        };
        t.show();
        System.out.println("=====");
    }
}
```

OutPut :

```
===== INNER CLASS DEMONSTRATION =====
This is Member Inner Class
This is Local Inner Class
This is Anonymous Inner Class
=====
```

Name : Patel Yashkumar Shaileshkumar

Enrollment No : 12402080601158

➤ Program 8

Problem Statement :

Create custom exception handling for bank withdrawal scenario

Objective:

To develop a Java program that demonstrates custom exception handling for a bank withdrawal scenario, ensuring proper handling of errors such as insufficient balance during withdrawal operations.

Source Code :

```
import java.util.Scanner;
class InsufficientBalanceException extends Exception {
    public InsufficientBalanceException(String msg) {
        super(msg);
    }
}
class BankAccount {
    int balance = 5000;
    void withdraw(int amount) throws InsufficientBalanceException {
        if (amount > balance) {
            throw new InsufficientBalanceException("Error : Insufficient Balance");
        } else {
            balance = balance - amount;
            System.out.println("-----");
            System.out.println("Withdrawal Successful");
            System.out.println("Remaining Balance : " + balance);
            System.out.println("-----");
        }
    }
}
public class Program8 {
    public static void main(String args[]) {

        Scanner sc = new Scanner(System.in);
        BankAccount b = new BankAccount();
        System.out.println("=====");
        System.out.println("                BANK SYSTEM                ");
        System.out.println("=====");
        System.out.println("Current Balance : 5000");
        System.out.print("Enter Amount to Withdraw : ");
        int amount = sc.nextInt();
        try {
            b.withdraw(amount);
        }
        catch (InsufficientBalanceException e) {
            System.out.println("-----");
            System.out.println(e.getMessage());
            System.out.println("Withdrawal Failed");
            System.out.println("-----");
        }
        System.out.println("Transaction Completed");
        System.out.println("=====");
    }
}
```

Output :

```
=====
                        BANK SYSTEM
=====
Current Balance : 5000
Enter Amount to Withdraw : 2000
-----
Withdrawal Successful
Remaining Balance : 3000
-----
Transaction Completed
=====
```